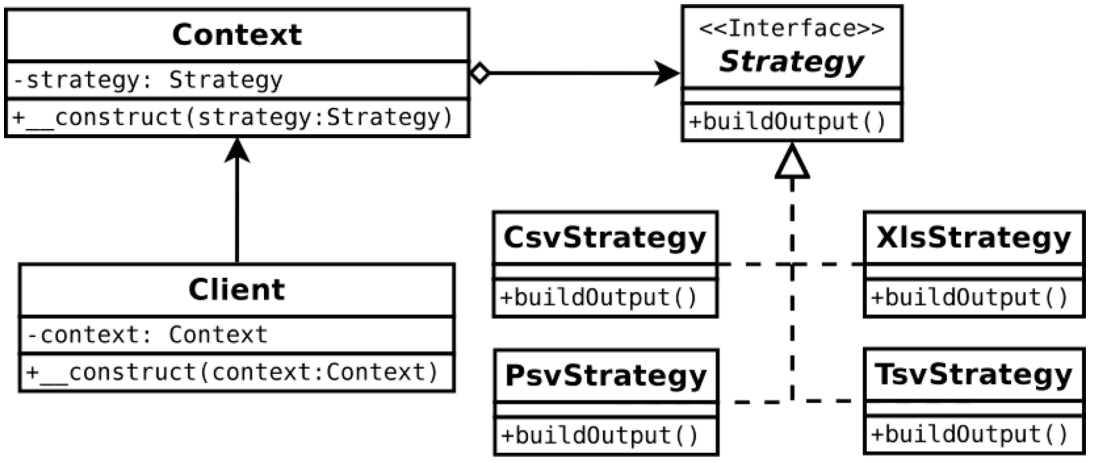


Figure 10-21. *Each layer points toward an abstraction*

The `Client` instance is given a `Context` instance via constructor injection (it's perfectly fine for client code to make its own `Context` or have it set after instantiation, but constructor injection is all the rage these days); in turn, the `Context` instance is created with a concrete implementation of a `Strategy` interface. How this is done is up for grabs, and naturally a creational pattern of some kind can handle the work for you (good patterns always integrate well with each other). It's likely that *somewhere* there's a logical construct working on the incoming `format` query parameter, but the key point is that it is not the `Client` instance doing it. Concerns have been separated as they should be. The Strategy Pattern is quite a common thing to see in modern PHP. Runtime algorithm selection is a ubiquitous problem, and this pattern is Tried and Tested™.

As before, here is a list of well-known behavioral patterns for you to read up on at your own convenience:

- Chain of Responsibility (notify handler objects in a chain, each one deciding whether to process or pass to the next)
- Command (package an event/request a stand-alone object containing all necessary information for processing)
- Iterator (traverse a collection without exposing the underlying structure, e.g., list, tree, etc.)
- Mediator (reduce dependency between closely related components by delegating interactions to a single component)